

Quantitative Finance Pricing Engines

(Supplements)

Neel Dev Sen

July 2026

Contents

1	Closed Form Solutions	4
1.1	Black-76 PDE	4
1.1.1	Derivation of the Black-76 PDE	4
1.1.2	Implementation of the Black-76 Struct	6
1.1.3	Black-76 Formula for Call Options	7
1.1.4	Black-76 Formula for Put Options	8
1.2	Merton's Dividend-Adjusted Black-Scholes	9
1.2.1	Derivation of Merton's Dividend-Adjusted Black-Scholes	9
1.2.2	Implementation of the Black-Scholes Merton Struct	12
1.2.3	Closed-Form Solution to Call Options with Dividends	12
1.2.4	Closed-Form Solution to Put Options with Dividends	14
2	Monte Carlo Simulations	15
2.1	Monte Carlo Simulation for Black 76	15
2.1.1	Derivation of the value of a Future	15
2.1.2	Monte Carlo Simulations for B76 Call Options	16
2.1.3	Monte Carlo Simulations for B76 Put Options	17
2.2	Monte Carlo Simulation with Dividends	19
2.2.1	Derivation of the value of a Stock with Dividends	19
2.2.2	Monte Carlo Simulations for Call Options with Dividends	20
2.2.3	Monte Carlo Simulations for Put Options with Dividends	22
3	Binomial Trees	23
3.1	Binomial Trees with Black-76 Options	23
3.1.1	Deriving the Risk-Neutral Probability for Futures	23
3.1.2	Binomial Tree for BS Call Options	26
3.1.3	Binomial Tree for BS Put Options	27
3.2	Binomial Trees for Stocks with Dividends	28
3.2.1	Binomial Tree for Dividend-Adjusted Call Options	30
3.2.2	Binomial Tree for Dividend-Adjusted Put Options	31
4	Trinomial Trees	32
4.1	Trinomial Trees with Black-76	32
4.1.1	Deriving the Risk-Neutral Probabilities	32
4.1.2	Implementation of the Trinomial Tree for B76 Call Options	34
4.1.3	Implementation of the Trinomial Tree for B76 Put Options	35
4.2	Trinomial Trees for Stocks with Dividends	36

4.2.1	Deriving the Risk-Neutral Probabilities	36
4.2.2	Implementation of the Trinomial Tree for Merton Call Options	37
4.2.3	Implementation of the Trinomial Tree for Merton Put Options	38

1 Closed Form Solutions

1.1 Black-76 PDE

The Black-76 model is just an adaption of the **Black-Scholes equation** except it uses the price of futures and bonds to price options.

A **future** is a contract where the buyer/seller agrees to purchase/sell an asset at a later fixed time

In **GitHub** the source file can be found here: **source file**

For the listings in this section I will continue using the header file **functions.hpp** that I defined on day 1. I will continue to use the **normalCDF** function from there.

1.1.1 Derivation of the Black-76 PDE

The modelling assumptions are that the futures of forward price process is modelled by **geometric Brownian motion** shown by the equation:

$$dF_t = \sigma F_t dW_t \quad (1.1)$$

where F_t is the price of a future or a forward at a given time, dF_t is the infinitesimal change of the price of a future of a forward, σ is implied volatility and dW_t is the infinitesimal change in the **Wiener function**. [4]

Substituting Equation 1.1 into **Ito's Lemma ??**

$$\begin{aligned} df(t, S_t) &= \frac{\partial f}{\partial t} dt + \frac{\partial f}{\partial F_t} (\sigma F_t dW_t) + \frac{1}{2} \frac{\partial^2 f}{\partial F_t^2} (\sigma F_t dW_t)^2 \\ &= \frac{\partial f}{\partial t} dt + \frac{\partial f}{\partial F_t} (\sigma F_t dW_t) + \frac{1}{2} \frac{\partial^2 f}{\partial F_t^2} (\sigma^2 F_t^2 (dW_t)^2) \end{aligned} \quad (1.2)$$

Substituting Equations ?? into Equation 1.2 we get:

$$\begin{aligned} df(t, S_t) &= \frac{\partial f}{\partial t} dt + \frac{\partial f}{\partial F_t} (\sigma F_t) dW_t + \frac{1}{2} \frac{\partial^2 f}{\partial F_t^2} (\sigma^2 F_t^2) dt \\ &= \left(\frac{\partial f}{\partial t} + \frac{1}{2} \sigma^2 F_t^2 \frac{\partial^2 f}{\partial F_t^2} \right) dt + \sigma F_t \frac{\partial f}{\partial F_t} dW_t \end{aligned} \quad (1.3)$$

Define

$$V(t, S) \equiv f(t, S_t) \quad (1.4)$$

Using the result from Equation 1.3

$$dV(t, S) = \left(\frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 S_t^2 \frac{\partial^2 V}{\partial F^2} \right) dt + \sigma F \frac{\partial V}{\partial F} dW_t \quad (1.5)$$

The thing that is different about the **delta-hedged portfolio** is that at the time the option is being exchanged, the cost of the **future** is 0.

$$\Pi = V \quad (1.6)$$

For an infinitesimal time step, the value of the future increases by dF which leads to

$$d\Pi = dV - \Delta dF \quad (1.7)$$

where $\Delta = \frac{\partial V}{\partial F}$ Substituting Equation 1.5 for dV and Equation 1.1 for dF we get.

$$\begin{aligned} d\Pi &= \left(\frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 F^2 \frac{\partial^2 V}{\partial F^2} \right) dt + \sigma F \frac{\partial V}{\partial F} dW_t - \Delta (\sigma F dW_t) \\ &= \left(\frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 F^2 \frac{\partial^2 V}{\partial F^2} \right) dt + \left(\sigma F \frac{\partial V}{\partial F} - \Delta \sigma F \right) dW_t \end{aligned} \quad (1.8)$$

Since $\Delta = \frac{\partial V}{\partial F}$, Equation 1.8 becomes:

$$\begin{aligned} d\Pi &= \left(\frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 F^2 \frac{\partial^2 V}{\partial F^2} \right) dt + \left(\sigma F \frac{\partial V}{\partial F} - \sigma F \frac{\partial V}{\partial F} \right) dW_t \\ &= \left(\frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 F^2 \frac{\partial^2 V}{\partial F^2} \right) dt \end{aligned} \quad (1.9)$$

Using Equation ?? and substituting this into Equation 1.9 we get:

$$\begin{aligned} r\Pi dt &= \left(\frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 F^2 \frac{\partial^2 V}{\partial F^2} \right) dt \\ r\Pi &= \frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 F^2 \frac{\partial^2 V}{\partial F^2} \end{aligned} \quad (1.10)$$

Substituting Equation 1.6 into Equation 1.10 we get:

$$rV = \frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 F^2 \frac{\partial^2 V}{\partial F^2} \quad (1.11)$$

Which finally leads to the equation:

$$\boxed{\frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 F^2 \frac{\partial^2 V}{\partial F^2} - rV = 0} \quad (1.12)$$

Which is also known as the **Black-76 Equation**

1.1.2 Implementation of the Black-76 Struct

Just like the Black-Scholes equation, the **Black-76** equation has its own struct since it uses **futures/forwards** instead of **stocks**. I will need to save the initial value of the future instead of a stock.

```
1  #ifndef BLACK76
2  #define BLACK76
3  #include <type_traits>
4
5  template <typename F_t, typename K_t, typename r_t, typename sigma_t,
6  ↪ typename T_t>
7
8  struct OptionDataB76
9  {
10     using commonType = std::common_type_t <double, F_t, K_t, r_t,
11     ↪ sigma_t, T_t>;
12     commonType future {};
13     commonType strike {};
14     commonType rate {};
15     commonType volatility {};
16     commonType maturity {};
17 };
18
19 template <typename F_t, typename K_t, typename r_t, typename sigma_t,
20 ↪ typename T_t>
21 OptionDataB76(F_t, K_t, r_t, sigma_t, T_t) -> OptionDataB76<F_t, K_t,
22 ↪ r_t, sigma_t, T_t>;
23
24 #endif
```

Listing 1.1: Black 76 Option Data Struct

This struct **OptionDataB76** is almost the same as the struct **OptionDataBS** except it passes a future instead of a stock. This means the type placeholder also changed from **S_t** for stocks to **F_t** for futures. The first data member the struct saves is **future** instead of **spot**. This struct is saved on the header: **black76-structs.hpp**

1.1.3 Black-76 Formula for Call Options

The Black-76 Formula for Call options is:

$$\begin{aligned} C(t, F) &= e^{-r(T-t)}(F\Phi(d_1) - K\Phi(d_2)) \\ d_1 &= \frac{\ln(\frac{F}{K}) + \frac{1}{2}\sigma^2(T-t)}{\sigma\sqrt{T-t}} \\ d_2 &= d_1 - \sigma\sqrt{T-t} \end{aligned} \tag{1.13}$$

[4] The implementation in C++ is:

```
1      #include <cmath>
2      #include <type_traits>
3      #include "../Headers/functions.hpp"
4      #include "../Headers/black76_structs.hpp"
5
6      template <typename F_t, typename K_t, typename r_t, typename sigma_t,
7      ↪   typename T_t>
8      auto black76Call(const OptionDataB76<F_t, K_t, r_t, sigma_t, T_t>&
9      ↪   data) -> decltype(data.future)
10     {
11         using commonType = decltype(data.future);
12         auto sqrt_maturity {std::sqrt(data.maturity)};
13
14         auto d1 {(std::log(data.future / data.strike) +
15         ↪   (static_cast<commonType>(0.5) * data.volatility *
16         ↪   data.volatility) * data.maturity ) / (data.volatility*
17         ↪   sqrt_maturity)};
18         auto d2 {d1 - data.volatility * sqrt_maturity};
19
20         auto C {(std::exp(-data.rate * data.maturity)) * (data.future *
21         ↪   normalCDF(d1) - data.strike * normalCDF(d2))};
22         return C;
23     }
```

Listing 1.2: Black-76 Call using Closed Form Solution

d_1 is calculated in line 12, d_2 is calculated in line 13 and finally Equation 1.13 is computed in line 15 (stored as **C**) and returned in line 16.

1.1.4 Black-76 Formula for Put Options

The Black-76 Formula for Call options is:

$$\begin{aligned}
 P(t, F) &= e^{-r(T-t)} (K\Phi(-d_2) - F\Phi(-d_1)) \\
 d_1 &= \frac{\ln(\frac{F}{K}) + \frac{1}{2}\sigma^2(T-t)}{\sigma\sqrt{T-t}} \\
 d_2 &= d_1 - \sigma\sqrt{T-t}
 \end{aligned}
 \tag{1.14}$$

[4] The implementation in C++ is:

```

1      #include <cmath>
2      #include <type_traits>
3      #include "../Headers/functions.hpp"
4      #include "../Headers/black76_structs.hpp"
5
6      template <typename F_t, typename K_t, typename r_t, typename sigma_t,
7      ↪   typename T_t>
8      auto black76Put(const OptionDataB76<F_t, K_t, r_t, sigma_t, T_t>& data)
9      ↪   -> decltype(data.future)
10     {
11         using commonType = decltype(data.future);
12         auto sqrt_maturity {std::sqrt(data.maturity)};
13
14         auto d1 {(std::log(data.future / data.strike) +
15         ↪   (static_cast<commonType>(0.5) * data.volatility *
16         ↪   data.volatility) * data.maturity ) / (data.volatility*
17         ↪   sqrt_maturity)};
18         auto d2 {d1 - data.volatility * sqrt_maturity};
19
20         auto P {std::exp(-data.rate * data.maturity) * (data.strike *
21         ↪   normalCDF(-d2) - (data.future * normalCDF(-d1)))};
22         return P;
23     }

```

Listing 1.3: Black-76 Call using Closed Form Solution

d_1 is calculated in line 12, d_2 is calculated in line 13 and finally Equation 1.14 is computed in line 15 (stored as **P**) and returned in line 16.

1.2 Merton's Dividend-Adjusted Black-Scholes

A **dividend** is cash payment from companies to shareholders for ownership in their stock. The key property with dividends is that each time a dividend is given to the shareholder, the stock price per share drops by that dividend amount and that the dividend grows as the stock grows.

The first subsection is a derivation of the **Merton's Dividend Adjust Black-Scholes PDE**, followed by two subsections on the closed form of the call option and on the closed form of the put option.

Both of these have my implementations in **C++**. In **GitHub** the source file can be found here: [source file](#)
as well as the header file: [functions.hpp](#)

1.2.1 Derivation of Merton's Dividend-Adjusted Black-Scholes

Let's define the dividend yield to be q . For each infinitesimal step in the stock price S_t we pay a dividend of value $qS_t dt$. Putting this into the adjusted **geometric Brownian motion** give us:

$$\begin{aligned} dS_t &= rS_t dt + \sigma S_t dW_t - qS_t dt \\ &= (r - q)S_t dt + \sigma S_t dW_t \end{aligned} \tag{1.15}$$

where the other variables defined here are the same defined in Equation ???. The derivation here is again almost identical to that of the **Black-Scholes equation**. Substituting Equation 1.15 for dS_t we get:

$$\begin{aligned} df(t, S_t) &= \frac{\partial f}{\partial t} dt + \frac{\partial f}{\partial S_t} ((r - q)S_t dt + \sigma S_t dW_t) + \frac{1}{2} \frac{\partial^2 f}{\partial S_t^2} ((r - q)S_t dt + \sigma S_t dW_t)^2 \\ &= \frac{\partial f}{\partial t} dt + \frac{\partial f}{\partial S_t} ((r - q)S_t dt + \sigma S_t dW_t) + \frac{1}{2} \frac{\partial^2 f}{\partial S_t^2} ((r - q)^2 S_t^2 (dt)^2 + 2r\sigma S_t^2 (dt)(dW_t) + \sigma^2 S_t^2 (dW_t)^2) \end{aligned} \tag{1.16}$$

Substituting Equations ??, ??, and ?? into Equation 1.16 we get:

$$\begin{aligned} df(t, S_t) &= \frac{\partial f}{\partial t} dt + \frac{\partial f}{\partial S_t} ((r - q)S_t dt + \sigma S_t dW_t) + \frac{1}{2} \frac{\partial^2 f}{\partial S_t^2} (0 + 0 + \sigma^2 S_t^2 (dt)) \\ &= \frac{\partial f}{\partial t} dt + \frac{\partial f}{\partial S_t} ((r - q)S_t dt + \sigma S_t dW_t) + \frac{1}{2} \frac{\partial^2 f}{\partial S_t^2} \sigma^2 S_t^2 \\ &= \left(\frac{\partial f}{\partial t} + (r - q)S_t \frac{\partial f}{\partial S_t} + \frac{1}{2} \sigma^2 S_t^2 \frac{\partial^2 f}{\partial S_t^2} \right) dt + \sigma S_t \frac{\partial f}{\partial S_t} dW_t \end{aligned} \tag{1.17}$$

Define

$$V(t, S) \equiv f(t, S_t) \quad (1.18)$$

Using the result from Equation 1.17

$$dV(t, S) = \left(\frac{\partial V}{\partial t} + (r - q)S \frac{\partial V}{\partial S} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} \right) dt + \sigma S \frac{\partial V}{\partial S} dW_t \quad (1.19)$$

Now we will define a delta-hedged portfolio Π and $\Delta = \frac{\partial V}{\partial S}$ that satisfies:

$$\Pi = V - \Delta S \quad (1.20)$$

For an infinitesimal time step, since we hedge the option by shorting the stock, we must issue dividends with yield q which is represented in an infinitesimal time step as: $-\Delta q S dt$ this becomes:

$$d\Pi = dV - \Delta dS - \Delta q S dt \quad (1.21)$$

Substituting Equation 1.19 for dV and Equation ?? for dS we get.

$$\begin{aligned} d\Pi &= \left(\frac{\partial V}{\partial t} + (r - q)S \frac{\partial V}{\partial S} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} \right) dt + \sigma S \frac{\partial V}{\partial S} dW_t - \Delta((r - q)S dt + \sigma S dW_t) - \Delta q S dt \\ &= \left(\frac{\partial V}{\partial t} + (r - q)S \frac{\partial V}{\partial S} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} - \Delta(r - q)S - \Delta q S \right) dt + (\sigma S \frac{\partial V}{\partial S} - \Delta \sigma S) dW_t \end{aligned} \quad (1.22)$$

Since $\Delta = \frac{\partial V}{\partial S}$, Equation 1.22 becomes:

$$\begin{aligned} d\Pi &= \left(\frac{\partial V}{\partial t} + (r - q)S \frac{\partial V}{\partial S} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} - (r - q)S \frac{\partial V}{\partial S} - qS \frac{\partial V}{\partial S} \right) dt + (\sigma S \frac{\partial V}{\partial S} - \sigma S \frac{\partial V}{\partial S}) dW_t \\ &= \left(\frac{\partial V}{\partial t} + (r - q)S \frac{\partial V}{\partial S} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} - (r - q)S \frac{\partial V}{\partial S} - qS \frac{\partial V}{\partial S} \right) dt \\ &= \left(\frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} - qS \frac{\partial V}{\partial S} \right) dt \end{aligned} \quad (1.23)$$

Substituting Equation ?? into Equation 1.23 we get:

$$\begin{aligned} r\Pi dt &= \left(\frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} - qS \frac{\partial V}{\partial S} \right) dt \\ r\Pi &= \frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} - qS \frac{\partial V}{\partial S} \end{aligned} \quad (1.24)$$

Substituting Equation 1.20 into Equation 1.24 we get:

$$\begin{aligned}
r(V - \Delta S) &= \frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} - qS \frac{\partial V}{\partial S} \\
r(V - \frac{\partial V}{\partial S} S) &= \frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} - qS \frac{\partial V}{\partial S} \\
rV - rS \frac{\partial V}{\partial S} &= \frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} - qS \frac{\partial V}{\partial S}
\end{aligned} \tag{1.25}$$

This finally gives us the equation:

$$\boxed{\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + (r - q)S \frac{\partial V}{\partial S} - rV = 0} \tag{1.26}$$

which is the **Black-Scholes PDE with Merton's dividend adjustment**. [5]

This PDE uses the same variables as the **Black-Scholes PDE** with the only addition being the **dividend yield** q . q affects the equation by modifying $rS \frac{\partial V}{\partial S}$ (Black-Scholes PDE) into the new $(r - q)S \frac{\partial V}{\partial S}$

1.2.2 Implementation of the Black-Scholes Merton Struct

```

1      #include <type_traits>
2      #ifndef MERTONSTRUCTS
3      #define MERTONSTRUCTS
4
5      template <typename S_t, typename K_t, typename r_t, typename q_t,
6              ↪ typename sigma_t, typename T_t>
7
8      struct OptionDataMerton
9      {
10         using CommonType = std::common_type_t <double, S_t, K_t, r_t, q_t,
11             ↪ sigma_t, T_t>;
12         CommonType spot {};
13         CommonType strike {};
14         CommonType rate {};
15         CommonType dividend {};
16         CommonType volatility {};
17         CommonType maturity {};
18     };
19
20     template <typename S_t, typename K_t, typename r_t, typename q_t,
21             ↪ typename sigma_t, typename T_t>
22     OptionDataMerton(S_t, K_t, r_t, q_t, sigma_t, T_t) ->
23     ↪ OptionDataMerton<S_t, K_t, r_t, q_t, sigma_t, T_t>;
24 #endif

```

Listing 1.4: Black-Scholes Merton Option Data Struct

This struct **OptionDataMerton** is very similar to both of the ones above, however we now also have the dividend member. The type placeholder for the dividend is `q_t` and it is initialised right after rate. This struct can be found in the header file: **merton-struct.hpp**

1.2.3 Closed-Form Solution to Call Options with Dividends

$$\begin{aligned}
 C(t, S) &= S e^{-q(T-t)} \Phi(d_1) - K e^{-r(T-t)} \Phi(d_2) \\
 d_1 &= \frac{\ln(\frac{S}{K}) + (r - q + \frac{1}{2}\sigma^2)(T - t)}{\sigma\sqrt{T - t}} \\
 d_2 &= d_1 - \sigma\sqrt{T - t}
 \end{aligned} \tag{1.27}$$

[5] The only thing that changed from Equation ?? is the d_1 term has a $-q$ term as well as the fact that in $C(t, S)$, the S term has an $e^{-q(T-t)}$ term multiplied to it.

Implementation in C++

```

1      #include <cmath>
2      #include <type_traits>
3      #include "../Headers/functions.hpp"
4      #include "../Headers/merton_struct.hpp"
5
6      template <typename S_t, typename K_t, typename r_t, typename q_t,
7      ↪   typename sigma_t, typename T_t>
8      auto bsDividendCall(const OptionDataMerton<S_t, K_t, r_t, q_t, sigma_t,
9      ↪   T_t>& data) -> decltype(data.spot)
10     {
11         using CommonType = decltype(data.spot);
12         auto sqrt_maturity {std::sqrt(data.maturity)};
13
14         auto d1 {(std::log(data.spot / data.strike) + (data.rate -
15         ↪   data.dividend + (static_cast<CommonType>(0.5) * data.volatility
16         ↪   * data.volatility)) * (data.maturity)) / (data.volatility *
17         ↪   sqrt_maturity)};
18         auto d2 {d1 - data.volatility * sqrt_maturity};
19
20         auto C {data.spot * std::exp(-data.dividend * data.maturity) *
21         ↪   normalCDF(d1) - data.strike * std::exp(-data.rate *
22         ↪   data.maturity) * normalCDF(d2)};
23         return C;
24     }

```

Listing 1.5: Black-76 Call using Closed Form Solution

This function is similar to the call function made in day 1, however the struct **OptionDataMerton** also has the data member **dividend**. The addition of **data.dividend** affects line 12 with the $(r - q)$ term instead of r . The addition also affects line 15 where the value of S is affected with $Se^{-q(T-t)}$ versus S . This function evaluates Equation 1.27 and stores it as **C** with a type of at least **double** and then returns it in line 16.

1.2.4 Closed-Form Solution to Put Options with Dividends

$$\begin{aligned}
 P(t, S) &= Ke^{-r(T-t)}\Phi(-d_2) - Se^{-q(T-t)}\Phi(-d_1) - \\
 d_1 &= \frac{\ln(\frac{S}{K}) + (r - q + \frac{1}{2}\sigma^2)(T - t)}{\sigma\sqrt{T - t}} \\
 d_2 &= d_1 - \sigma\sqrt{T - t}
 \end{aligned} \tag{1.28}$$

[5] Again just like with the call, the only changes from Equation ?? is the fact that the d_1 term has a $-q$ term as well as the fact that in $P(t, S)$, the S term has an $e^{-q(T-t)}$ term multiplied to it.

Implementation in C++

```

1  #include <cmath>
2  #include <type_traits>
3  #include "../Headers/functions.hpp"
4  #include "../Headers/merton_struct.hpp"
5
6  template <typename S_t, typename K_t, typename r_t, typename q_t,
7  ↪   typename sigma_t, typename T_t>
8  auto bsDividendPut(const OptionDataMerton<S_t, K_t, r_t, q_t, sigma_t,
9  ↪   T_t>& data) -> decltype(data.spot)
10 {
11     using CommonType = decltype(data.spot);
12     auto sqrt_maturity {std::sqrt(data.maturity)};
13
14     auto d1 {(std::log(data.spot / data.strike) + (data.rate -
15     ↪   data.dividend + (static_cast<CommonType>(0.5) * data.volatility
16     ↪   * data.volatility)) * (data.maturity)) / (data.volatility *
17     ↪   sqrt_maturity)};
18     auto d2 {d1 - data.volatility * sqrt_maturity};
19
20     auto P {data.strike * std::exp(-data.rate * data.maturity) *
21     ↪   normalCDF(-d2) - data.spot * std::exp(-data.dividend *
22     ↪   data.maturity) * normalCDF(-d1)};
23     return P;
24 }
```

Listing 1.6: Black-76 Call using Closed Form Solution

Function got the same changes as in listing 1.5 where it calculates d_1 in line 12, d_2 in line 13 and computes Equation 1.28 in line 15 and returns it in line 18.

2 Monte Carlo Simulations

2.1 Monte Carlo Simulation for Black 76

The source file can be found on my **GitHub**

2.1.1 Derivation of the value of a Future

First we will use the Black-76 assumption defined in Equation 1.1

$$\begin{aligned}dF_t &= \sigma F_t dW_t \\ \frac{dF_t}{F_t} &= \sigma dW_t\end{aligned}$$

Define:

$$X_t = \ln(F_t) \tag{2.1}$$

Using **Ito's Lemma** we can represent dX_t in terms of dF_t like this:

$$dX_t = \frac{dX_t}{dF_t} dF_t + \frac{1}{2} \frac{d^2 X_t}{dF_t^2} (dF_t)^2$$

Since $\frac{dX_t}{dF_t} = \frac{1}{F_t}$ and $\frac{d^2 X_t}{dF_t^2} = -\frac{1}{(F_t)^2}$. Substituting this in the previous equation we get:

$$dX_t = \frac{1}{F_t} (\sigma F_t dW_t) - \frac{1}{2F_t^2} (\sigma F_t dW_t)^2 \tag{2.2}$$

Using **Ito's multiplication rules** defined in Equation ?? , ?? and ??, we can simplify the equation to:

$$\begin{aligned}dX_t &= \sigma dW_t - \frac{1}{2F_t^2} (\sigma^2 F_t^2 dt) \\ dX_t &= \sigma dW_t - \frac{1}{2} \sigma^2 dt\end{aligned} \tag{2.3}$$

Again by introducing a dummy variable s we can finally integrate all of the terms:

$$\begin{aligned}\int_t^T dX_s &= \int_t^T \sigma dW_s - \int_t^T \frac{1}{2} \sigma^2 ds \\ X_T - X_t &= \sigma(W_T - W_t) - \frac{1}{2} \sigma^2 (T - t)\end{aligned} \tag{2.4}$$

Substituting $\ln(F_T) = X_T$ and $\ln(F_t) = X_t$ into Equation 2.4 we get:

$$\begin{aligned}\ln(F_T) - \ln(F_t) &= \sigma(W_T - W_t) - \frac{1}{2}\sigma(T - t) \\ \ln\left(\frac{F_T}{F_t}\right) &= \sigma(W_T - W_t) - \frac{1}{2}\sigma(T - t) \\ \frac{F_T}{F_t} &= \exp(\sigma(W_T - W_t) - \frac{1}{2}\sigma(T - t))\end{aligned}\tag{2.5}$$

This leaves us with the equation of a stock which is:

$$\boxed{F_T = F_t \exp(\sigma(W_T - W_t) - \frac{1}{2}\sigma(T - t))}\tag{2.6}$$

2.1.2 Monte Carlo Simulations for B76 Call Options

In order to price an option with the Monte Carlo Simulation we must use the call payoff definition of an option. A payoff for a call is defined as:

$$\text{Payoff} = \max(F_T - K, 0)\tag{2.7}$$

Where S_T is the stock price at the time of maturity T

When we use a Monte Carlo Simulation we simulate a lot of these values for S_T to find the expected value of a payoff. After we find the expected value of a payoff, we must discount it using the risk-free rate r like this:

$$\boxed{C = e^{-r(T-t)}\mathbb{E}(\max(F_T - K, 0))}\tag{2.8}$$

where $e^{-r(T-t)}$ is the discount factor and F_T is the same as defined in Equation 2.6 In order to simulate the Wiener function, we use Equation ??

```

1      #include <cmath>
2      #include <random>
3      #include <algorithm>
4      #include "../Headers/black76_structs.hpp"
5
6      template <typename F_t, typename K_t, typename r_t, typename sigma_t,
7      ↪   typename T_t>
8      auto monteCarloB76Call(const OptionDataB76<F_t, K_t, r_t, sigma_t,
9      ↪   T_t>& data, int simulations) -> decltype(data.future)
10     {
11         using commonType = decltype(data.future);
12         std::random_device rd{};
13         std::mt19937 mt{rd()};
14         std::normal_distribution W {static_cast<commonType>(0),
15         ↪   std::sqrt(data.maturity)};
16         commonType payoffs {};
17
18         for (int i {}; i < simulations; ++i)
19         {
20             commonType F_final {data.future * std::exp(((data.volatility *
21             ↪   W(mt)) - 0.5 * data.volatility * data.volatility *
22             ↪   data.maturity))};
23             payoffs += std::max(F_final - data.strike,
24             ↪   static_cast<commonType>(0));
25         }
26         payoffs /= simulations;
27
28         return std::exp(-data.rate * data.maturity) * payoffs;
29     }

```

Listing 2.1: Black-76 Call using Monte Carlo Simulations

This function is really similar to that of the one defined in Listing ?? when it comes to the random number generation. This function passes in the **OptionDataB76** Struct instead as well as the integer parameter **simulations**. It computes Equation ?? in line 18 and takes the average, discounts it and returns the value of the call option in line 22.

2.1.3 Monte Carlo Simulations for B76 Put Options

For a put option, it is pretty much the opposite, the payoff for a put option is defined as:

$$\text{Payoff} = \max(K - F_T, 0) \quad (2.9)$$

The equation for the put option value at time t is represented as:

$$P = e^{-r(T-t)} \mathbb{E}(\max(K - F_T, 0)) \quad (2.10)$$

where $e^{-r(T-t)}$ is the discount factor and F_T is the same as defined in Equation 2.6

```

1      #include <cmath>
2      #include <random>
3      #include <algorithm>
4      #include "../Headers/black76_structs.hpp"
5
6      template <typename F_t, typename K_t, typename r_t, typename sigma_t,
7      ↪   typename T_t>
8      auto monteCarloB76Put(const OptionDataB76<F_t, K_t, r_t, sigma_t, T_t>&
9      ↪   data, int simulations) -> decltype(data.future)
10     {
11         using commonType = decltype(data.future);
12         std::random_device rd{};
13         std::mt19937 mt{rd{}};
14         std::normal_distribution W {static_cast<commonType>(0),
15         ↪   std::sqrt(data.maturity)};
16         commonType payoffs {};
17
18         for (int i {}; i < simulations; ++i)
19         {
20             commonType F_final {data.future * std::exp(((data.volatility *
21             ↪   W(mt)) - 0.5 * data.volatility * data.volatility *
22             ↪   data.maturity))};
23             payoffs += std::max(data.strike - F_final,
24             ↪   static_cast<commonType>(0));
25         }
26         payoffs /= simulations;
27
28         return std::exp(-data.rate * data.maturity) * payoffs;
29     }

```

Listing 2.2: Black-76 Put using Monte Carlo Simulations

This implementation is almost identical except in line 18 the payoffs evaluate Equation 2.9 instead of Equation 2.7

2.2 Monte Carlo Simulation with Dividends

The source can again be found on my **GitHub**

2.2.1 Derivation of the value of a Stock with Dividends

First we will use the **Merton SDE** defined in Equation 1.15

$$\begin{aligned} dS_t &= (r - q)S_t dt + \sigma S_t dW_t \\ \frac{dS_t}{S_t} &= (r - q) dt + \sigma dW_t \end{aligned}$$

Define:

$$X_t = \ln(S_t) \quad (2.11)$$

Using **Ito's Lemma** we can represent dX_t in terms of dS_t like this:

$$dX_t = \frac{dX_t}{dS_t} dS_t + \frac{1}{2} \frac{d^2 X_t}{dS_t^2} (dS_t)^2$$

Since $\frac{dX_t}{dS_t} = \frac{1}{S_t}$ and $\frac{d^2 X_t}{dS_t^2} = -\frac{1}{(S_t)^2}$. Substituting this in the previous equation we get:

$$\begin{aligned} dX_t &= \frac{1}{S_t} ((r - q)S_t dt + \sigma S_t dW_t) - \frac{1}{2S_t^2} ((r - q)S_t dt + \sigma S_t dW_t)^2 \\ dX_t &= (r - q) dt + \sigma dW_t - \frac{1}{2S_t^2} ((r - q)^2 S_t^2 (dt)^2 + 2r\sigma S_t^2 dt dW_t + \sigma^2 S_t^2 (dW_t)^2) \end{aligned} \quad (2.12)$$

Using **Ito's multiplication rules** defined in Equation ??, ??, and ??, we can simplify the equation to:

$$\begin{aligned} dX_t &= (r - q) dt + \sigma dW_t - \frac{1}{2S_t^2} (\sigma^2 S_t^2 dt) \\ dX_t &= \left(r - q - \frac{1}{2}\sigma^2 \right) dt + \sigma dW_t \end{aligned} \quad (2.13)$$

Introducing a dummy variable s we can finally integrate all of the terms:

$$\begin{aligned} \int_t^T dX_s &= \int_t^T \left(r - q - \frac{1}{2}\sigma^2 \right) ds + \int_t^T \sigma dW_s \\ X_T - X_t &= \left(r - q - \frac{1}{2}\sigma^2 \right) (T - t) + \sigma(W_T - W_t) \end{aligned} \quad (2.14)$$

Substituting $\ln(S_T) = X_T$ and $\ln(S_t) = X_t$ into Equation ?? we get:

$$\begin{aligned}\ln(S_T) - \ln(S_t) &= \left(r - q - \frac{1}{2}\sigma^2\right)(T - t) + \sigma(W_T - W_t) \\ \ln\left(\frac{S_T}{S_t}\right) &= \left(r - q - \frac{1}{2}\sigma^2\right)(T - t) + \sigma(W_T - W_t) \\ \frac{S_T}{S_t} &= \exp\left(\left(r - q - \frac{1}{2}\sigma^2\right)(T - t) + \sigma(W_T - W_t)\right)\end{aligned}\tag{2.15}$$

This leaves us with the equation of a stock which is:

$$\boxed{S_T = S_t \exp\left(\left(r - q - \frac{1}{2}\sigma^2\right)(T - t) + \sigma(W_T - W_t)\right)}\tag{2.16}$$

2.2.2 Monte Carlo Simulations for Call Options with Dividends

In order to price an option with the Monte Carlo Simulation we must use the call payoff definition of an option. A payoff for a call is defined as:

$$\text{Payoff} = \max(S_T - K, 0)\tag{2.17}$$

Where S_T is the stock price with dividends at the time of maturity T .

The value of the call option is equal to:

$$\boxed{C = e^{-r(T-t)}\mathbb{E}(\max(S_T - K, 0))}\tag{2.18}$$

where $e^{-r(T-t)}$ is the discount factor and S_T is the same as defined in Equation 2.16

Implementing the call option pricer in C++ like this:

```
1      #include <cmath>
2      #include <random>
3      #include <algorithm>
4      #include "../Headers/merton_struct.hpp"
5
6      template <typename S_t, typename K_t, typename r_t, typename q_t,
7      ↪   typename sigma_t, typename T_t>
8      auto monteCarloMertonCall(const OptionDataMerton<S_t, K_t, r_t, q_t,
9      ↪   sigma_t, T_t>& data, int simulations) -> decltype(data.spot)
10     {
11         using commonType = decltype(data.spot);
12         std::random_device rd{};
13         std::mt19937 mt{rd()};
14         std::normal_distribution W {static_cast<commonType>(0),
15         ↪   std::sqrt(data.maturity)};
16         commonType payoffs {};
17
18         for (int i {}; i < simulations; ++i)
19         {
20             commonType S_final {data.spot * std::exp((data.rate -
21             ↪   data.dividend - 0.5 * data.volatility * data.volatility) *
22             ↪   data.maturity + (data.volatility * W(mt)))};
23             payoffs += std::max(S_final - data.strike,
24             ↪   static_cast<commonType>(0));
25         }
26         payoffs /= simulations;
27
28         return std::exp(-data.rate * data.maturity) * payoffs;
29     }
```

Listing 2.3: Black-Scholes -Merton Call using Monte Carlo Simulations

This function **monteCarloMertonCall** is extremely similar to the one from the GBM Monte Carlo simulation except for the fact it introduces another parameter in the struct **OptionDataMerton** which is the data member **dividend**. This modified line 18 with the modification to Equation 2.17 which then influences the value being returned in line 22 representing Equation 2.18.

2.2.3 Monte Carlo Simulations for Put Options with Dividends

For a put option, it again is pretty much the opposite, with the payoff for a put option being defined as:

$$\text{Payoff} = \max(K - S_T, 0) \quad (2.19)$$

The value of a put option is represented as:

$$P = e^{-r(T-t)} \mathbb{E}(\max(K - S_T, 0)) \quad (2.20)$$

where S_T is the same as defined in Equation 2.16

```

1  #include <cmath>
2  #include <random>
3  #include <algorithm>
4  #include "../Headers/merton_struct.hpp"
5
6  template <typename S_t, typename K_t, typename r_t, typename q_t,
7  ↪   typename sigma_t, typename T_t>
8  auto monteCarloMertonPut(const OptionDataMerton<S_t, K_t, r_t, q_t,
9  ↪   sigma_t, T_t>& data, int simulations) -> decltype(data.spot)
10 {
11     using commonType = decltype(data.spot);
12     std::random_device rd{};
13     std::mt19937 mt{rd()};
14     std::normal_distribution W {static_cast<commonType>(0),
15     ↪   std::sqrt(data.maturity)};
16     commonType payoffs {};
17
18     for (int i {}; i < simulations; ++i)
19     {
20         commonType S_final {data.spot * std::exp((data.rate -
21         ↪   data.dividend - 0.5 * data.volatility * data.volatility) *
22         ↪   data.maturity + (data.volatility * W(mt)))};
23         payoffs += std::max(data.strike - S_final,
24         ↪   static_cast<commonType>(0));
25     }
26     payoffs /= simulations;
27
28     return std::exp(-data.rate * data.maturity) * payoffs;
29 }

```

Listing 2.4: Black-Scholes-Merton Put using Monte Carlo Simulations

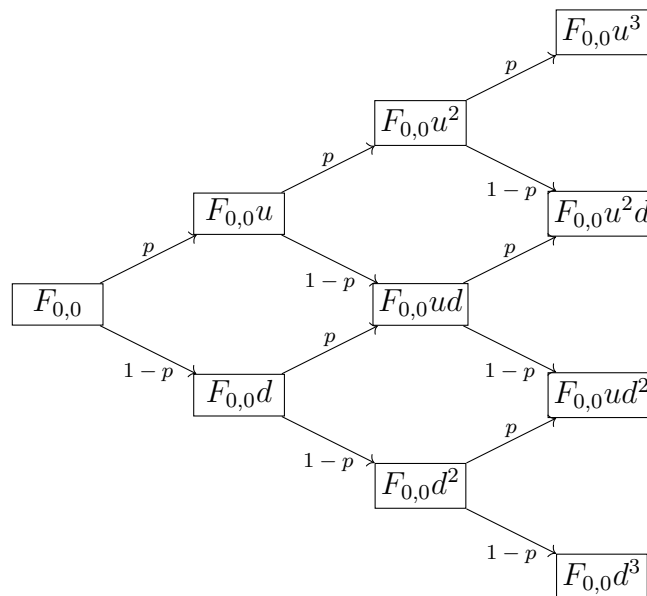
This implementation is almost identical except in line 18 the payoffs evaluate Equation 2.19 instead of Equation 2.17

3 Binomial Trees

3.1 Binomial Trees with Black-76 Options

3.1.1 Deriving the Risk-Neutral Probability for Futures

A Binomial Tree is a numerical method that is produced by creating values until the time to maturity. The nodes at maturity are all the payoffs, and with the binomial tree we work backwards to find the price of the option at the initial time.



each step = Δt

Figure 1: Binomial Tree for the future price

This is what a binomial tree for a future looks like. It can go up with probability p , and go down with probability $1 - p$. The stock $F_{a,b}$ is the node that is up a nodes and down b nodes

$$F_{a,b} = u^a d^b F_{0,0} \quad (3.1)$$

In order to derive the values for u and d we are going to use the equation of a stock from Equation 2.6

$$F_T = F_t \exp(\sigma(W_T - W_t) - \frac{1}{2}\sigma^2(T - t))$$

On an interval of time Δt we have

$$\begin{aligned} \frac{F_{t+\Delta t}}{F_t} &= \frac{F_t \exp((\sigma(W_T - W_t) - \frac{1}{2}\sigma^2(T - t)))}{F_t} \\ \frac{F_{t+\Delta t}}{F_t} &= \exp(\sigma(W_T - W_t) - \frac{1}{2}\sigma^2(T - t)) \end{aligned}$$

Using equation ?? we have:

$$\begin{aligned} W_{t+\Delta t} - W_t &\sim \mathcal{N}(0, \Delta t) \\ W_{t+\Delta t} - W_t &\sim \Delta t(\mathcal{N}(0, 1)) \end{aligned}$$

Letting $Z \sim \mathcal{N}(0, 1)$ we have:

$$\frac{F_{t+\Delta t}}{F_t} = \exp((\sigma(\sqrt{\Delta t}Z - \frac{1}{2}\sigma^2(T - t)))$$

At very small Δt , $\sqrt{\Delta t} \gg \Delta t$ which means this equation can be approximated to:

$$\frac{F_{t+\Delta t}}{F_t} \cong \exp(\sigma\sqrt{\Delta t}Z) \quad (3.2)$$

Using the same method for the Black-Scholes by defining a normal variable Z_{bin} we solve for u like this:

$$\begin{aligned} u &= \frac{F_{t+\Delta t}}{F_t} \\ u &= e^{\sigma\sqrt{\Delta t}Z_{bin}} \end{aligned}$$

Substituting $Z_{bin} = 1$ we get the value of u to be:

$$\boxed{u = e^{\sigma\sqrt{\Delta t}}} \quad (3.3)$$

And for d like this:

$$\begin{aligned} d &= \frac{F_{t+\Delta t}}{F_t} \\ d &= e^{\sigma\sqrt{\Delta t}Z_{bin}} \end{aligned}$$

Now substituting $Z_{bin} = -1$ we get the value of d to be:

$$\boxed{d = e^{-\sigma\sqrt{\Delta t}}} \quad (3.4)$$

When deriving the risk-neutral probability we know that the expected value of a stock at time $t + \Delta t$ is the value of that of t compounded for the time Δt or:

$$\mathbb{E}[F_{t+\Delta t}] = F_t \quad (3.5)$$

According to the binomial tree

$$\begin{aligned} \mathbb{E}[F_{t+\Delta t}] &= F_t(up + (1-p)d) \\ F_t &= F_t(up + (1-p)d) \\ 1 &= up + (1-p)d \\ 1 &= up + d - dp \\ 1 &= (u-d)p + d \\ \frac{e^{r\Delta t} - d}{u - d} &= p \end{aligned}$$

Substituting Equations ?? and ?? for u and d respectively we get the risk neutral probability p to be:

$$\boxed{p = \frac{1 - e^{-\sigma\sqrt{\Delta t}}}{e^{\sigma\sqrt{\Delta t}} - e^{-\sigma\sqrt{\Delta t}}}} \quad (3.6)$$

We will again do the same thing as we did for the Black-Scholes by computing the terminal payoffs and using a backward recursion to calculate the value of the option using this formula for the recursion:

$$V_{i,j} = e^{-r\Delta t}(pV_{i+1,j+1} + (1-p)V_{i+1,j}) \quad (3.7)$$

In order to create the terminal payoffs for a Binomial Tree of size n , the formula for a European Call is:

$$V_{n,k} = \max(u^k d^{n-k} F_{0,0} - K, 0) \quad (3.8)$$

where $0 \leq k \leq n$

For a put European Put option, the terminal payoffs are:

$$V_{k,n-k} = \max(K - u^k d^{n-k} F_{0,0}, 0) \quad (3.9)$$

3.1.2 Binomial Tree for BS Call Options

```

1  #include <cmath>
2  #include <vector>
3  #include <type_traits>
4  #include <algorithm>
5  #include "../Headers/black76_structs.hpp"
6
7  template <typename F_t, typename K_t, typename r_t, typename sigma_t,
8  ↪   typename T_t>
9  auto binomialTreeB76Call(const OptionDataB76<F_t, K_t, r_t, sigma_t, T_t>&
10 ↪   data, const int N) -> decltype(data.future)
11 {
12     using commonType = decltype(data.future);
13
14     auto sqrt_deltat {std::sqrt(data.maturity /
15 ↪   static_cast<commonType>(N))};
16
17     auto u {std::exp(data.volatility * sqrt_deltat)};
18     auto d {std::exp(-data.volatility * sqrt_deltat)};
19     auto p {(1 - d) / (u - d)};
20
21     std::vector<commonType> V_current (N + 1);
22     for (int j {}; j < N + 1; ++j)
23     {
24         V_current[j] = std::max(data.future *
25 ↪   std::pow(static_cast<double>(u) , static_cast<double>(j)) *
26 ↪   std::pow(static_cast<double>(d), static_cast<double>(N-j)) -
27 ↪   data.strike , static_cast<commonType>(0));
28     }
29
30     for (int i {}; i < N ; ++i)
31     {
32         std::vector<commonType> V_new (N-i);
33         for (int k {}; k < N-i; ++k)
34         {
35             V_new[k] = p * V_current[k+1] + (1-p) * V_current[k];
36         }
37         std::swap(V_current, V_new);
38     }
39
40     return V_current[0] * std::exp(-data.rate * data.maturity);
41 }

```

Listing 3.1: Black-Scholes-Merton Put using Monte Carlo Simulations

3.1.3 Binomial Tree for BS Put Options

```

1  #include <cmath>
2  #include <vector>
3  #include <type_traits>
4  #include <algorithm>
5  #include "../Headers/black76_structs.hpp"
6
7  template <typename F_t, typename K_t, typename r_t, typename sigma_t,
8  ↪   typename T_t>
9  auto binomialTreeB76Put(const OptionDataB76<F_t, K_t, r_t, sigma_t, T_t>&
10 ↪   data, const int N) -> decltype(data.future)
11 {
12     using commonType = decltype(data.future);
13     auto sqrt_deltat {std::sqrt(data.maturity /
14 ↪   static_cast<commonType>(N))};
15
16     auto u {std::exp(data.volatility * sqrt_deltat)};
17     auto d {std::exp(-data.volatility * sqrt_deltat)};
18     auto p {(1 - d) / (u - d)};
19
20     std::vector<commonType> V_current (N + 1);
21     for (int j {}; j < N + 1; ++j)
22     {
23         V_current[j] = std::max(data.strike - data.future *
24 ↪   std::pow(static_cast<double>(u) , static_cast<double>(j)) *
25 ↪   std::pow(static_cast<double>(d), static_cast<double>(N-j)) ,
26 ↪   static_cast<commonType>(0));
27     }
28
29     for (int i {}; i < N ; ++i)
30     {
31         std::vector<commonType> V_new (N-i);
32         for (int k {}; k < N-i; ++k)
33         {
34             V_new[k] = p * V_current[k+1] + (1-p) * V_current[k];
35         }
36         std::swap(V_current, V_new);
37     }
38     return V_current[0] * std::exp(-data.rate * data.maturity);
39 }

```

Listing 3.2: Black-Scholes-Merton Put using Monte Carlo Simulations

The only thing that is different about these 2 functions rather than that for **GBM** is the fact that they pass in the **OptionDataB76** struct
Line 16 it uses the formula from Equation 3.6 instead of Equation ??.
Line 21 uses the formula in Equation 3.8 and 3.9 respectively.

3.2 Binomial Trees for Stocks with Dividends

This is again very similar to the Black Scholes GBM, just accounting for dividends.
In order to derive the values for u and d we are going to use the equation of a stock from Equation 2.16

$$S_T = S_t \exp\left((r - q - \frac{1}{2}\sigma^2)(T - t) + \sigma(W_T - W_t)\right)$$

On an interval of time Δt we have

$$\begin{aligned} \frac{S_{t+\Delta t}}{S_t} &= \frac{S_t \exp\left((r - q - \frac{1}{2}\sigma^2)(\Delta t) + \sigma(W_{t+\Delta t} - W_t)\right)}{S_t} \\ \frac{S_{t+\Delta t}}{S_t} &= \exp\left((r - q - \frac{1}{2}\sigma^2)(\Delta t) + \sigma(W_{t+\Delta t} - W_t)\right) \end{aligned}$$

Using equation ?? we have:

$$\begin{aligned} W_{t+\Delta t} - W_t &\sim \mathcal{N}(0, \Delta t) \\ W_{t+\Delta t} - W_t &\sim \Delta t(\mathcal{N}(0, 1)) \end{aligned}$$

Letting $Z \sim \mathcal{N}(0, 1)$ we have:

$$\frac{S_{t+\Delta t}}{S_t} = \exp\left((r - q - \frac{1}{2}\sigma^2)(\Delta t) + \sigma\sqrt{\Delta t}Z\right)$$

At very small Δt , $\sqrt{\Delta t} \gg \Delta t$ which means this equation can be approximated to:

$$\frac{S_{t+\Delta t}}{S_t} \cong \exp(\sigma\sqrt{\Delta t}Z) \tag{3.10}$$

Again for the 3rd time we define a Binomial Normal Variable Z_{bin} such that $\Pr(Z_{bin} = 1) = \frac{1}{2}$ and $\Pr(Z_{bin} = -1) = \frac{1}{2}$

We define the up factor u as the factor when $Z_{bin} = 1$ and down factor d when $Z_{bin} = -1$

$$u = \frac{S_{t+\Delta t}}{S_t}$$

$$u = e^{\sigma\sqrt{\Delta t}Z_{bin}}$$

Substituting $Z_{bin} = 1$ we get the value of u to be:

$$\boxed{u = e^{\sigma\sqrt{\Delta t}}} \quad (3.11)$$

Doing the same for the down factor:

$$d = \frac{S_{t+\Delta t}}{S_t}$$

$$d = e^{\sigma\sqrt{\Delta t}Z_{bin}}$$

Now substituting $Z_{bin} = -1$ we get the value of d to be:

$$\boxed{d = e^{-\sigma\sqrt{\Delta t}}} \quad (3.12)$$

The only thing different than the expected value of a stock using the **GBM** assumption is the dividend yield adjustment which is shown like this:

$$\mathbb{E}[S_{t+\Delta t}] = S_t e^{(r-q)\Delta t} \quad (3.13)$$

According to the binomial tree

$$\begin{aligned} \mathbb{E}[S_{t+\Delta t}] &= S_t(up + (1-p)d) \\ S_t e^{(r-q)\Delta t} &= S_t(up + (1-p)d) \\ e^{(r-q)\Delta t} &= up + (1-p)d \\ e^{(r-q)\Delta t} &= up + d - dp \\ e^{(r-q)\Delta t} &= (u-d)p + d \\ \frac{e^{(r-q)\Delta t} - d}{u-d} &= p \end{aligned}$$

Substituting Equations 3.11 and 3.12 for u and d respectively we get the risk neutral probability p to be:

$$\boxed{p = \frac{e^{(r-q)\Delta t} - e^{-\sigma\sqrt{\Delta t}}}{e^{\sigma\sqrt{\Delta t}} - e^{-\sigma\sqrt{\Delta t}}}} \quad (3.14)$$

3.2.1 Binomial Tree for Dividend-Adjusted Call Options

```

1  #include <cmath>
2  #include <vector>
3  #include <type_traits>
4  #include <algorithm>
5  #include "../Headers/merton_struct.hpp"
6
7  template <typename S_t, typename K_t, typename r_t, typename q_t, typename
8  ↪ sigma_t, typename T_t>
9  auto binomialTreeMertonCall(const OptionDataMerton<S_t, K_t, r_t, q_t,
10 ↪ sigma_t, T_t>& data, const int N) -> decltype(data.spot)
11 {
12     using commonType = decltype(data.spot);
13     auto sqrt_deltat {std::sqrt(data.maturity /
14 ↪ static_cast<commonType>(N))};
15
16     auto u {std::exp(data.volatility * sqrt_deltat)};
17     auto d {std::exp(-data.volatility * sqrt_deltat)};
18     auto p {(std::exp((data.rate - data.dividend) * data.maturity / N) - d)
19 ↪ / (u - d)};
20
21     std::vector<commonType> V_current (N + 1);
22     for (int j {}; j < N + 1; ++j)
23     {
24         V_current[j] = std::max(data.spot * std::pow(static_cast<double>(u)
25 ↪ , static_cast<double>(j)) * std::pow(static_cast<double>(d),
26 ↪ static_cast<double>(N-j)) - data.strike ,
27 ↪ static_cast<commonType>(0));
28     }
29
30     for (int i {}; i < N ; ++i)
31     {
32         std::vector<commonType> V_new (N-i);
33         for (int k {}; k < N-i; ++k)
34         {
35             V_new[k] = p * V_current[k+1] + (1-p) * V_current[k];
36         }
37         std::swap(V_current, V_new);
38     }
39
40     return V_current[0] * std::exp(-data.rate * data.maturity);
41 }

```

Listing 3.3: Binomial tree Calls for the Merton dividend model

3.2.2 Binomial Tree for Dividend-Adjusted Put Options

```

1  #include <cmath>
2  #include <vector>
3  #include <type_traits>
4  #include <algorithm>
5  #include "../Headers/merton_struct.hpp"
6
7  template <typename S_t, typename K_t, typename r_t, typename q_t, typename
8  ↪ sigma_t, typename T_t>
9  auto binomialTreeMertonPut(const OptionDataMerton<S_t, K_t, r_t, q_t,
10 ↪ sigma_t, T_t>& data, const int N) -> decltype(data.spot)
11 {
12     using commonType = decltype(data.spot);
13
14     auto sqrt_deltat {std::sqrt(data.maturity /
15 ↪ static_cast<commonType>(N))};
16
17     auto u {std::exp(data.volatility * sqrt_deltat)};
18     auto d {std::exp(-data.volatility * sqrt_deltat)};
19     auto p {(std::exp((data.rate - data.dividend) * data.maturity / N) - d)
20 ↪ / (u - d)};
21
22     std::vector<commonType> V_current (N + 1);
23     for (int j {}; j < N + 1; ++j)
24     {
25         V_current[j] = std::max(data.strike - data.spot *
26 ↪ std::pow(static_cast<double>(u) , static_cast<double>(j)) *
27 ↪ std::pow(static_cast<double>(d), static_cast<double>(N-j)) ,
28 ↪ static_cast<commonType>(0));
29     } u^{k-1}d^{n-k+1}
30
31     for (int i {}; i < N ; ++i)
32     {
33         std::vector<commonType> V_new (N-i);
34         for (int k {}; k < N-i; ++k)
35         {
36             V_new[k] = p * V_current[k+1] + (1-p) * V_current[k];
37         }
38         std::swap(V_current, V_new);
39     }
40     return V_current[0] * std::exp(-data.rate * data.maturity);
41 }

```

Listing 3.4: Binomial tree Puts for the Merton dividend model

The only thing that changed from the GBM Assumption pricer is that this function passes in the struct **OptionDataMerton** instead and in line 16 represents the formula in Equation 3.14.

4 Trinomial Trees

4.1 Trinomial Trees with Black-76

4.1.1 Deriving the Risk-Neutral Probabilities

Since in the previous subsection we found that $u_{tri} = u_{bi}^2$ and $d_{tri} = d_{bi}^2$. We substitute the values of $u_{bi} = e^{\sigma\sqrt{\Delta t}}$ and $d_{bi} = e^{-\sigma\sqrt{\Delta t}}$ for the Black-76 model to get that

$$\boxed{u_{tri} = e^{2\sigma\sqrt{\Delta t}}} \quad (4.1)$$

$$\boxed{d_{tri} = e^{-2\sigma\sqrt{\Delta t}}} \quad (4.2)$$

$$\boxed{m = 1} \quad (4.3)$$

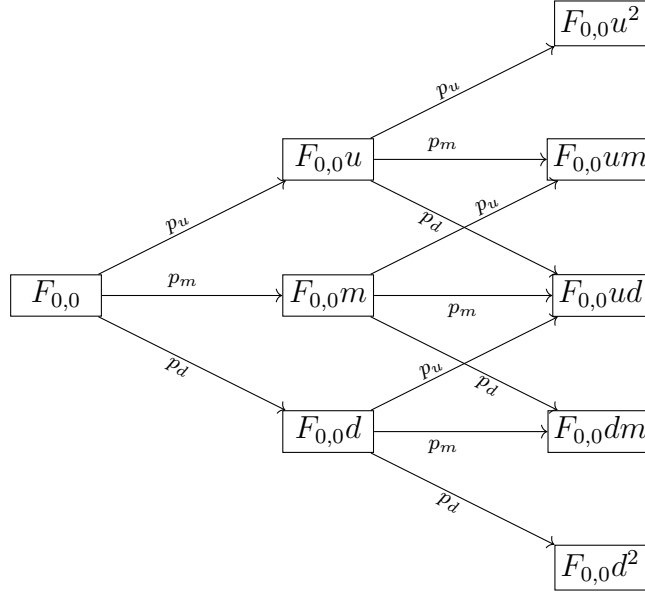
Which just like the Binomial tree, the Black-76 up and down factors are identical to that of the Black-Scholes. What won't be the same is the risk-neutral probabilities. Again using $p = \frac{1 - e^{-\sigma\sqrt{\Delta t}}}{e^{\sigma\sqrt{\Delta t}} - e^{-\sigma\sqrt{\Delta t}}}$ as the risk-neutral probability for the Binomial tree derived in Equation 3.6, and the fact that $p_u = p^2$ and $p_d = (1 - p)^2$ we get:

$$\boxed{p_u = \left(\frac{1 - e^{-\sigma\sqrt{\Delta t}}}{e^{\sigma\sqrt{\Delta t}} - e^{-\sigma\sqrt{\Delta t}}} \right)^2} \quad (4.4)$$

$$\boxed{p_d = \left(\frac{e^{-\sigma\sqrt{\Delta t}} - 1}{e^{\sigma\sqrt{\Delta t}} - e^{-\sigma\sqrt{\Delta t}}} \right)^2} \quad (4.5)$$

Again since all of the probabilities must add up to 1, p_m is equal to:

$$\boxed{p_m = 1 - p_u - p_d} \quad (4.6)$$



each step = $2\Delta t$

Figure 2: Trinomial Tree for the future price

Using the same notation for the Trinomial tree that was used for the Black-Scholes, a future at any node is given with the equation:

$$F_{i,j} = u^j F_{0,0} \quad (4.7)$$

When calculating the payoffs, Equation ?? still applies. The only other thing that changes using Equation 4.7 when trying to calculate the value of the terminal payoffs. For a call option, the terminal payoff is given as:

$$V_{n,k} = \max(u^j F_{0,0} - K, 0) \quad (4.8)$$

4.1.2 Implementation of the Trinomial Tree for B76 Call Options

```

1  #include <cmath>
2  #include <vector>
3  #include <type_traits>
4  #include <algorithm>
5  #include "../Headers/black76_structs.hpp"
6
7  template <typename F_t, typename K_t, typename r_t, typename sigma_t,
8  ↪   typename T_t>
9  auto trinomialTreeB76Call(const OptionDataB76<F_t, K_t, r_t, sigma_t, T_t>&
10 ↪   data, const int N) -> decltype(data.future)
11 {
12     using commonType = decltype(data.future);
13
14     auto sqrt_2deltat {std::sqrt(2 * data.maturity /
15 ↪   static_cast<commonType>(N))};
16
17     auto u {std::exp(data.volatility * sqrt_2deltat)};
18     auto d {std::exp(-data.volatility * sqrt_2deltat)};
19
20     auto p_u {std::pow((1 - std::exp(-data.volatility * sqrt_2deltat /
21 ↪   static_cast<commonType>(2))) / ((std::exp(data.volatility *
22 ↪   sqrt_2deltat / static_cast<commonType>(2))) -
23 ↪   std::exp(-data.volatility * sqrt_2deltat /
24 ↪   static_cast<commonType>(2))), static_cast<commonType>(2))};
25     auto p_d {std::pow((std::exp(data.volatility * sqrt_2deltat /
26 ↪   static_cast<commonType>(2)) - 1) / ((std::exp(data.volatility *
27 ↪   sqrt_2deltat / static_cast<commonType>(2))) -
28 ↪   std::exp(-data.volatility * sqrt_2deltat /
29 ↪   static_cast<commonType>(2))), static_cast<commonType>(2))};
30     auto p_m {static_cast<commonType>(1) - p_u - p_d};

```

Listing 4.1: Trinomial Tree for Black-76 Call Option (Part I)

This function called **trinomialTreeB76Call** is almost identical to the one for Black-Scholes Calls except it takes in a struct parameter called **OptionDataB76**. The function again uses N as the number of iterations.

Line 8 computes Equation 4.1 instead of ???. Similarly in line 9 it computes Equation 4.2 instead of Equation ???.

In line 11 it uses Equation 4.4 instead of Equation ??? in order to calculate the value of p_u just like in line 12 using Equation 4.5 instead of using Equation ??? to calculate p_d and finally in line 13 where it computes Equation 4.6 to get the value of p_m

```

14     std::vector<commonType> V_current (2 * N + 1);
15     for (int j {}; j < 2 * N + 1; ++j)
16     {
17         V_current[j] = std::max(data.future *
18             ↪ std::pow(static_cast<commonType>(u) ,
19             ↪ static_cast<commonType>(j- N)) - data.strike,
20             ↪ static_cast<commonType>(0));
21     }

```

Listing 4.2: Trinomial Tree for Black-76 Call Option (Part II)

The only thing that is different here is that in Line 17 it computes Equation 4.8 instead of Equation ??.

```

19     for (int i {}; i < N ; ++i)
20     {
21         std::vector<commonType> V_new (2 * N - 2 * i - 1);
22         for (int k {}; k < 2* N-2*i -1; ++k)
23         {
24             V_new[k] = p_u * V_current[k+2] + p_m * V_current[k+1] + p_d *
25                 ↪ V_current[k];
26         }
27         std::swap(V_current, V_new);
28     }
29     return V_current[0] * std::exp(-data.rate * data.maturity);

```

Listing 4.3: Trinomial Tree for Black-76 Call Option (Part III)

This part of the function is actually identical to that of the Black-Scholes Trinomial tree.

4.1.3 Implementation of the Trinomial Tree for B76 Put Options

$$V_{n,k} = \max(K - u^j F_{0,0}, 0) \quad (4.9)$$

```

14     std::vector<commonType> V_current (2 * N + 1);
15     for (int j {}; j < 2 * N + 1; ++j)
16     {
17         V_current[j] = std::max(data.strike - data.future *
18             ↪ std::pow(static_cast<commonType>(u) ,
19             ↪ static_cast<commonType>(j- N)), static_cast<commonType>(0));
20     }

```

Listing 4.4: Trinomial Tree for Black-76 Put Options (Part II)

The only thing that changed in this function (called **trinomialTreeB76Put**) is that in line 17 it computes Equation 4.9 instead of Equation 4.8.

4.2 Trinomial Trees for Stocks with Dividends

4.2.1 Deriving the Risk-Neutral Probabilities

Since in the previous subsection we found that $u_{tri} = u_{bi}^2$ and $d_{tri} = d_{bi}^2$. We substitute the values of $u_{bi} = e^{\sigma\sqrt{\Delta t}}$ and $d_{bi} = e^{-\sigma\sqrt{\Delta t}}$ for the Merton model to get that

$$\boxed{u_{tri} = e^{2\sigma\sqrt{\Delta t}}} \quad (4.10)$$

$$\boxed{d_{tri} = e^{-2\sigma\sqrt{\Delta t}}} \quad (4.11)$$

$$\boxed{m = 1} \quad (4.12)$$

Again just like the Binomial tree, the Merton up and down factors are in fact still identical to that of the Black-Scholes. What won't be the same is the risk-neutral probabilities. Using $p = \frac{e^{(r-q)\Delta t} - e^{-\sigma\sqrt{\Delta t}}}{e^{\sigma\sqrt{\Delta t}} - e^{-\sigma\sqrt{\Delta t}}}$ as the risk-neutral probability for the Binomial tree derived in Equation 3.14, and the fact that $p_u = p^2$ and $p_d = (1 - p)^2$ we get:

$$\boxed{p_u = \left(\frac{e^{(r-q)\Delta t} - e^{-\sigma\sqrt{\Delta t}}}{e^{\sigma\sqrt{\Delta t}} - e^{-\sigma\sqrt{\Delta t}}} \right)^2} \quad (4.13)$$

$$\boxed{p_d = \left(\frac{e^{\sigma\sqrt{\Delta t}} - e^{(r-q)\Delta t}}{e^{\sigma\sqrt{\Delta t}} - e^{-\sigma\sqrt{\Delta t}}} \right)^2} \quad (4.14)$$

Again since all of the probabilities must add up to 1, p_m is equal to:

$$\boxed{p_m = 1 - p_u - p_d} \quad (4.15)$$

The rest of the values for the Merton model are identical to that of Black-Scholes in term of syntax (If we did substitute the actual values for Equation ?? then yes it would be different.)

4.2.2 Implementation of the Trinomial Tree for Merton Call Options

```

1      #include <cmath>
2      #include <vector>
3      #include <type_traits>
4      #include <algorithm>
5      #include "../Headers/merton_struct.hpp"
6
7      template <typename S_t, typename K_t, typename r_t, typename q_t,
8      ↪   typename sigma_t, typename T_t>
9      auto trinomialTreeMertonCall(const OptionDataMerton<S_t, K_t, r_t, q_t,
10      ↪   sigma_t, T_t>& data, const int N) -> decltype(data.spot)
11      {
12          using commonType = decltype(data.spot);
13
14          auto sqrt_2deltat {std::sqrt(2 * data.maturity /
15      ↪   static_cast<commonType>(N))};
16
17          auto u {std::exp(data.volatility * sqrt_2deltat)};
18          auto d {std::exp(-data.volatility * sqrt_2deltat)};
19          auto p_u {std::pow((std::exp((data.rate - data.dividend) *
20      ↪   data.maturity / (2 * N)) - std::exp(-data.volatility *
21      ↪   sqrt_2deltat / static_cast<commonType>(2) )) /
22      ↪   ((std::exp(data.volatility * sqrt_2deltat /
23      ↪   static_cast<commonType>(2))) - std::exp(-data.volatility *
24      ↪   sqrt_2deltat / static_cast<commonType>(2))),
25      ↪   static_cast<commonType>(2))};
26
27          auto p_d {std::pow((std::exp(data.volatility * sqrt_2deltat /
28      ↪   static_cast<commonType>(2)) - std::exp((data.rate -
29      ↪   data.dividend) * data.maturity / (2 * N))) /
30      ↪   ((std::exp(data.volatility * sqrt_2deltat /
31      ↪   static_cast<commonType>(2))) - std::exp(-data.volatility *
32      ↪   sqrt_2deltat / static_cast<commonType>(2))),
33      ↪   static_cast<commonType>(2))};
34          auto p_m {static_cast<commonType>(1) - p_u - p_d};

```

Listing 4.5: Trinomial Tree for Merton Call Option (Part I)

This function called **trinomialTreeMertonCall** is really similar to the one for Black-Scholes Calls except it takes in a struct parameter called **OptionDataMerton** which has another data member **dividend**. The function again uses N as the number of iterations.

Line 8 computes Equation 4.10 instead of ???. Line 9 computes Equation 4.11 instead of Equation ???.

Line 11 uses Equation 4.13 instead of Equation 4.13 in order to calculate the value of p_u just like in Line 12 using Equation 4.14 instead of using Equation 4.14 to calculate p_d and finally in line 13 where it computes Equation 4.15 to get the value of p_m

```

14     std::vector<commonType> V_current (2 * N + 1);
15     for (int j {}; j < 2 * N + 1; ++j)
16     {
17         V_current[j] = std::max(data.spot *
18             ↪ std::pow(static_cast<commonType>(u) ,
19             ↪ static_cast<commonType>(j- N)) - data.strike,
20             ↪ static_cast<commonType>(0));
21     }
22
23     for (int i {}; i < N ; ++i)
24     {
25         std::vector<commonType> V_new (2 * N - 2 * i - 1);
26         for (int k {}; k < 2* N-2*i -1; ++k)
27         {
28             V_new[k] = p_u * V_current[k+2] + p_m * V_current[k+1] +
29             ↪ p_d * V_current[k];
30         }
31         std::swap(V_current, V_new);
32     }
33
34     return V_current[0] * std::exp(-data.rate * data.maturity);

```

Listing 4.6: Trinomial Tree for Merton Call Option (Part II + III)

This part of the function is actually identical to that of the Black-Scholes Trinomial tree.

4.2.3 Implementation of the Trinomial Tree for Merton Put Options

```

14     std::vector<commonType> V_current (2 * N + 1);
15     for (int j {}; j < 2 * N + 1; ++j)
16     {
17         V_current[j] = std::max(data.strike - data.future *
18             ↪ std::pow(static_cast<commonType>(u) ,
19             ↪ static_cast<commonType>(j- N)), static_cast<commonType>(0));
20     }

```

Listing 4.7: Trinomial Tree for Black-76 Put Options (Part II)

For puts the only thing that changed for this function (called **trinomial-TreeMetronPut**) was that Line 17 uses Equation ?? instead of ??.

References

- [1] Shreve, S. (2004). *Stochastic Calculus for Finance I: The Binomial Asset Pricing Model*. Springer Science & Business Media.
- [2] Lalley, S. P. (n.d.). *Brownian motion*. <https://galton.uchicago.edu/~lalley/Courses/313/BrownianMotionCurrent.pdf>
- [3] Haugh, M. (2016). *The Black-Scholes model* (IEOR E4706: Foundations of Financial Engineering lecture notes). Columbia University. <https://www.columbia.edu/~mh2078/FoundationsFE/BlackScholes.pdf>
- [4] Black, F. (1976). The pricing of commodity contracts. *Journal of Financial Economics*, 3(1–2), 167–179. [https://doi.org/10.1016/0304-405X\(76\)90024-6](https://doi.org/10.1016/0304-405X(76)90024-6)
- [5] Buchanan, J. R. (2014). *Extensions to the Black-Scholes equation: An undergraduate introduction to financial mathematics*. Millersville University. <https://sites.millersville.edu/rbuchanan/book/Extensions.pdf>